

Bachelor of Information Technology (Honours) Degree

Batch 02 – Level 04 – Semester 02

ECU4314– Data Science

Assignment 02

W.A.P.D. Hansika

October 2024

Table of Contents

Analysis Of Cardiac Data Of Suva Hospital Patients	1
INTRODUCTION	1
TASK 01	2
1. Adding Two Blank Spaces	2
2. Add Two Duplicate Records	2
3. Open Anaconda Software And Jupyter Notebook	3
4. Create New Folder And Rename It	3
5. Create New Python File	3
6. Upload The Csv File	3
7. Import Libraries	4
8. Read The Csv File	4
9. View The Dataset	4
10. Checking For Null Values	5
11. Fill For The Null Values	5
12. Again Checking For Null Values	5
13. Checking For The Duplicates And Remove Them	5
TASK 02	6
14. Import Matplotlib Library	6
15. Extract And Display Cholesterol Values	6
16. Extract And Display Cholesterol Values	6
17. Scatter Plot For Data Visualization	7
TASK 03	8
18. Import The Sklearn Libraries	8
19. Features And Target Variable	8
20. Split The Dataset	8
21. Initialize The Knn Classifier With 5 Neighbors	8
22. Train The Model On The Training Data	8
23. Predict On The Test Data	8
24. Evaluate Model Performance	8
25. Calculate Accuracy	8
26. Print Classification Report	9

27.	Print Confusion Matrix	9
TASK 04		9
28.	Adding New Data For Predication.....	9
29.	Predict Using The Trained Knn Model.....	9
30.	Plot The New Samples Alongside The Training Data	9
31.	Display The Predicted Results	10
FULL CODE		11

Analysis of Cardiac Data of Suva Hospital Patients Introduction

Cardiovascular diseases (CVD) remain one of the leading causes of death worldwide, and their early diagnosis is crucial for effective treatment and management. Advances in data analysis and machine learning have opened new avenues for understanding risk factors and improving early detection of heart diseases. In line with this, the WELLNESS Hospital has undertaken a study to analyze cardiac data from its patients. The dataset collected, "heart.csv," is a comprehensive resource detailing various cardiac-related attributes that could provide insights into potential risk factors and assist in predicting cardiac events.

This dataset focuses on multiple clinical and physiological measurements gathered from heart patients and presents valuable opportunities for researchers and healthcare professionals to uncover patterns in patient health data. By examining factors such as age, blood pressure, cholesterol levels, and ECG characteristics, this dataset enables advanced analysis, including predictive modeling, to support proactive health interventions.

The primary goal of collecting this dataset is to provide a foundation for analyzing key risk factors and clinical symptoms associated with heart disease. By leveraging statistical techniques and machine learning models, this data can be instrumental in developing diagnostic tools to support clinicians in identifying at-risk patients and designing personalized treatment plans.

age: The age of the patient.

sex: Gender of the patient (0: female, 1: male).

cp: Type of chest pain.

trestbps: Resting blood pressure.

chol: Serum cholesterol.

fbs: Fasting blood sugar > 120 mg/dl.

restecg: Resting electrocardiographic results.

thalach: Maximum heart rate achieved.

exang: Exercise induced angina.

oldpeak: ST depression induced by exercise relative to rest

Task 01

First edit the heart.csv file manually and add two blank spaces and two duplicate records. Then the heart.csv dataset should be cleaned. Also sample values can be given when filling in the blanks.

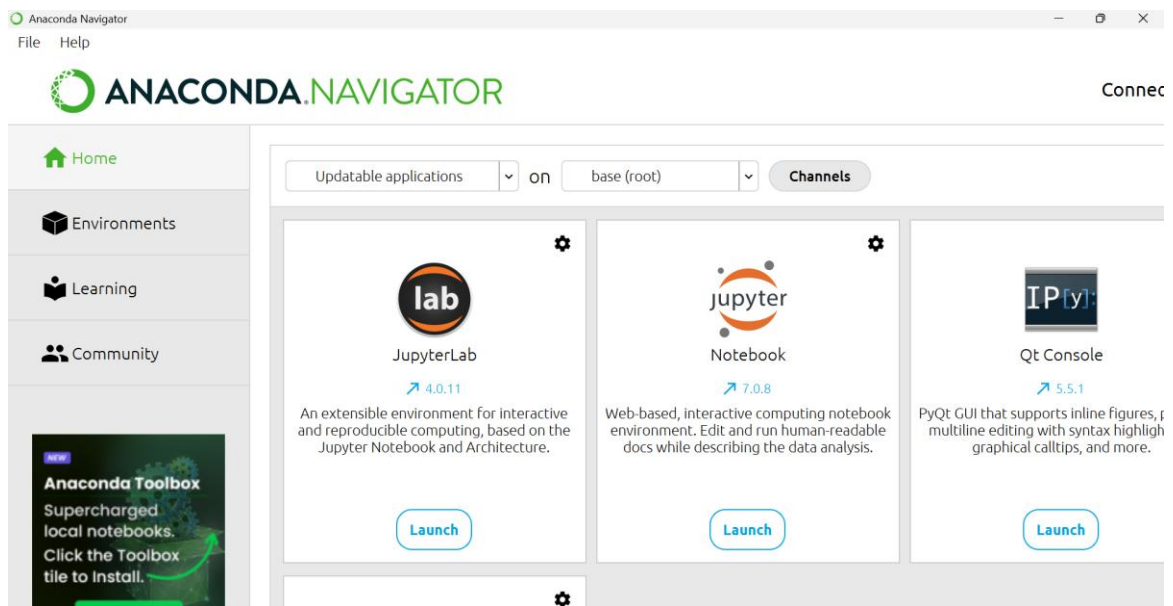
1. Adding Two Blank Spaces

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
2	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
3	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
4	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
5	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
6	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1
7	57	1	0	140	192	0	1	148	0	0.4	1	0	1	1
8	56	0	1	140	294	0	0	153	0	1.3	1	0	2	1
9	44	1	1	120	263	0	1	173	0	0	2	0	3	1
10	52	1	2	172		1	1	162	0	0.5	2	0	3	1
11	57	1	2	150	168	0	1	174	0	1.6	2	0	2	1
12	54	1	0	140	239	0	1	160	0	1.2	2	0	2	1
13	48	0	2	130	275	0	1	139	0	0.2	2	0	2	1
14	49	1	1	130	266	0	1	171	0	0.6	2	0	2	1
15	64	1	3	110	211	0	0	144	1	1.8	1	0	2	1
16	58	0	3	150	283		0	162	0	1	2	0	2	1
17	50	0	2	120	219	0	1	158	0	1.6	1	0	2	1
18	58	0	2	120	340	0	1	172	0	0	2	0	2	1
19	66	0	3	150	226	0	1	114	0	2.6	0	0	2	1
20	43	1	0	150	247	0	1	171	0	1.5	2	0	2	1

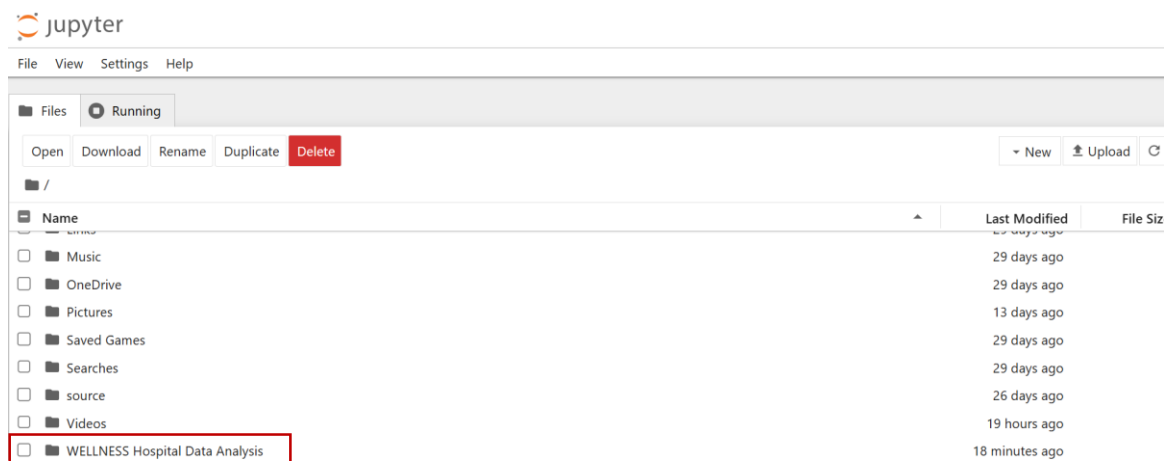
2. Add two duplicate records

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
2	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
3	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
4	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
5	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
6	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
7	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1
8	57	1	0	140	192	0	1	148	0	0.4	1	0	1	1
9	56	0	1	140	294	0	0	153	0	1.3	1	0	2	1
10	44	1	1	120	263	0	1	173	0	0	2	0	3	1
11	52	1	2	172		1	1	162	0	0.5	2	0	3	1
12	57	1	2	150	168	0	1	174	0	1.6	2	0	2	1
13	54	1	0	140	239	0	1	160	0	1.2	2	0	2	1
14	48	0	2	130	275	0	1	139	0	0.2	2	0	2	1
15	49	1	1	130	266	0	1	171	0	0.6	2	0	2	1
16	64	1	3	110	211	0	0	144	1	1.8	1	0	2	1
17	58	0	3	150	283		0	162	0	1	2	0	2	1
18	50	0	2	120	219	0	1	158	0	1.6	1	0	2	1
19	58	0	2	120	340	0	1	172	0	0	2	0	2	1
20	66	0	3	150	226	0	1	114	0	2.6	0	0	2	1

3. Open Anaconda Software and Jupyter Notebook



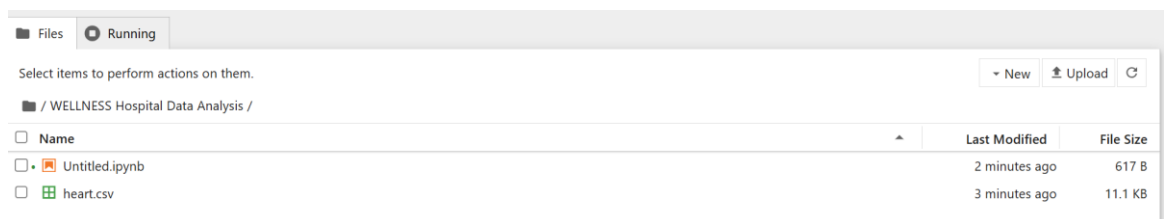
4. Create New Folder and Rename it.



5. Create New Python File



6. Upload the CSV File



Jupyter heart.csv Last Checkpoint: 26 minutes ago

File Edit View Settings Help

Delimiter: ,

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	
1	63	1	3	145	233	1	0	150	
2	63	1	3	145	233	1	0	150	
3	37	1	2	130	250	0	1	187	
4	41	0	1	130	204	0	0	172	
5	56	1	1	120	236	0	1	178	
6	57	0	0	120	354	0	1	163	
7	57	1	0	140	192	0	1	148	
8	56	0	1	140	294	0	0	153	
9	44	1	1	120	263	0	1	173	
10	52	1	2	172		1	1	162	
11	57	1	2	150	168	0	1	174	
12	54	1	0	140	239	0	1	160	
13	48	0	2	130	275	0	1	139	
14	49	1	1	130	266	0	1	171	
15	64	1	3	110	211	0	0	144	
16	58	0	3	150	283		0	162	
17	50	0	2	120	219	0	1	158	

7. Import Libraries

```
[2]: # Import Pandas Libraries
import pandas as pd
```

```
[4]: # Import Numpy Libraries
import numpy as np
```

8. Read the CSV File

```
[6]: # Load the Dataset
df=pd.read_csv("heart.csv")
```

9. View the Dataset

```
[8]: # View the Dataset
print(df)
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	\
0	63	1	3	145	233.0	1.0	0	150	0	2.3	
1	63	1	3	145	233.0	1.0	0	150	0	2.3	
2	37	1	2	130	250.0	0.0	1	187	0	3.5	
3	41	0	1	130	204.0	0.0	0	172	0	1.4	
4	56	1	1	120	236.0	0.0	1	178	0	0.8	
..	...	...	..	...	...	...	...	...	...	...	
299	57	0	0	140	241.0	0.0	1	123	1	0.2	
300	45	1	3	110	264.0	0.0	1	132	0	1.2	
301	68	1	0	144	193.0	1.0	1	141	0	3.4	
302	57	1	0	130	131.0	0.0	1	115	1	1.2	
303	57	0	1	130	236.0	0.0	0	174	0	0.0	

	slope	ca	thal	target
0	0	0	1	1
1	0	0	1	1
2	0	0	2	1
3	2	0	2	1
4	2	0	2	1
..	...	..	...	...
299	1	0	3	0
300	1	0	3	0
301	1	2	3	0
302	1	1	3	0
303	1	1	2	0

[304 rows x 14 columns]

10. Checking for Null Values

```
[10]: # Checking for Null Values  
df.isnull().sum()
```

```
[10]: age          0  
sex          0  
cp           0  
trestbps     0  
chol         1  
fbs          1  
restecg      0  
thalach      0  
exang        0  
oldpeak      0  
slope        0  
ca           0  
thal         0  
target       0  
dtype: int64
```

11. Fill For the Null Values

```
[12]: # Filling for Null Values  
df = df.fillna({'chol': 255, 'fbs': 1})
```

12. Again Checking for Null Values

```
[14]: # Again Checking Null Values  
df.isnull().sum()
```

```
[14]: age          0  
sex          0  
cp           0  
trestbps     0  
chol         0  
fbs          0  
restecg      0  
thalach      0  
exang        0  
oldpeak      0  
slope        0  
ca           0  
thal         0  
target       0  
dtype: int64
```

13. Checking for the Duplicates and Remove Them

```
[16]: # Checking for Duplicates  
df.duplicated().sum()
```

```
[16]: 2
```

```
[18]: # Removing Duplicates  
df = df.drop_duplicates()
```

```
[20]: # Again Checking Duplicates Values  
df.duplicated().sum()
```

```
[20]: 0
```


Task 02

Visualize the heart.csv dataset using an appropriate visualization technique.

14. Import Matplotlib Library

```
[22]: # Import Matplotlib
import matplotlib.pyplot as plt
```

15. Extract and Display Cholesterol Values

```
: # Extract cholesterol values from the 'chol' column as a NumPy array
x=df["chol"].values
```

```
: # Display the extracted cholesterol values
print(x)
```

```
[233. 250. 204. 236. 354. 192. 294. 263. 255. 168. 239. 275. 266. 211.
 283. 219. 340. 226. 247. 239. 234. 233. 226. 243. 199. 302. 212. 175.
 417. 197. 198. 177. 219. 273. 213. 177. 304. 232. 269. 360. 308. 245.
 208. 264. 321. 325. 235. 257. 216. 234. 256. 302. 231. 141. 252. 201.
 222. 260. 182. 303. 265. 309. 186. 203. 211. 183. 222. 234. 220. 209.
 258. 227. 204. 261. 213. 250. 245. 221. 205. 240. 250. 308. 318. 298.
 265. 564. 277. 197. 214. 248. 255. 207. 223. 288. 160. 226. 394. 233.
 315. 246. 244. 270. 195. 240. 196. 211. 234. 236. 244. 254. 325. 126.
 313. 211. 262. 215. 214. 193. 204. 243. 303. 271. 268. 267. 199. 210.
 204. 277. 196. 269. 201. 271. 295. 235. 306. 269. 178. 208. 201. 263.
 295. 303. 209. 223. 197. 245. 242. 240. 226. 180. 228. 149. 227. 278.
 220. 197. 253. 192. 220. 221. 240. 342. 157. 175. 286. 229. 268. 254.
 203. 256. 229. 284. 224. 206. 167. 230. 335. 177. 276. 353. 225. 330.
 230. 243. 290. 253. 266. 233. 172. 305. 216. 188. 282. 185. 326. 231.
 254. 267. 248. 197. 258. 270. 274. 164. 255. 239. 258. 188. 177. 229.
 260. 219. 307. 249. 341. 263. 330. 254. 256. 407. 217. 282. 288. 239.
 174. 281. 198. 288. 309. 243. 289. 289. 246. 322. 299. 300. 293. 304.
 282. 269. 249. 212. 274. 184. 274. 409. 246. 283. 254. 298. 247. 294.
 299. 273. 309. 259. 200. 244. 231. 228. 230. 282. 269. 206. 212. 327.
 149. 286. 283. 249. 234. 237. 234. 275. 212. 218. 261. 319. 166. 315.
 204. 218. 223. 207. 311. 204. 232. 335. 205. 203. 318. 225. 212. 169.
 187. 197. 176. 241. 264. 193. 131. 236.]
```

16. Extract and Display Cholesterol Values

```
[32]: # Extract resting blood pressure values from the 'trestbps' column as a NumPy array
y=df["trestbps"].values
```

```
[34]: # Display the extracted blood pressure values
print(y)
```

```
[145 130 130 120 120 140 140 120 172 150 140 130 130 110 150 120 120 150
 150 140 135 130 140 150 140 160 150 110 140 130 105 120 130 125 125 142
 135 150 155 160 140 130 104 130 140 120 140 138 128 138 130 120 130 108
 135 134 122 115 118 128 110 108 118 135 140 138 100 130 120 124 120 94
 130 140 122 135 125 140 128 105 112 128 102 152 102 115 118 101 110 100
 124 132 138 132 112 142 140 108 130 130 148 178 140 120 129 120 160 138
 120 110 180 150 140 110 130 120 130 120 105 138 130 138 112 108 94 118
 112 152 136 120 160 134 120 110 126 130 120 128 110 128 120 115 120 106
 140 156 118 150 120 130 160 112 170 146 138 130 130 122 125 130 120 132
 120 138 160 120 140 130 140 130 110 120 132 130 110 117 140 120 150 132
 150 130 112 150 112 130 124 140 110 130 128 120 145 140 170 150 125 120
 110 110 125 150 180 160 128 110 150 120 140 128 120 118 145 125 132 130
 130 135 130 150 140 138 200 110 145 120 120 170 125 108 165 160 120 130
 140 125 140 125 126 160 174 145 152 132 124 134 160 192 140 140 132 138
 100 160 142 128 144 150 120 178 112 123 108 110 112 180 118 122 130 120
 134 120 100 110 125 146 124 136 138 136 128 126 152 140 140 134 154 110
 128 148 114 170 152 120 140 124 164 140 110 144 130 130]
```

17. Scatter Plot for Data Visualization

```
[38]: # Create a scatter plot with cholesterol on the x-axis and resting blood pressure on the y-axis
plt.scatter(df['chol'], df['trestbps'])

# Add title and Labels to the plot
plt.title("Cholesterol vs Resting Blood Pressure")
plt.ylabel('Resting Blood Pressure')
plt.xlabel('Cholesterol')

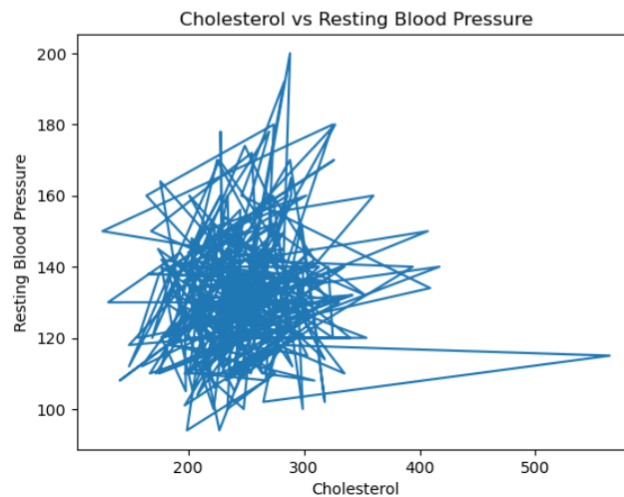
# Display the Scatter Plot for Data Visualization
plt.show()
```



```
[40]: # Create a plot with cholesterol on the x-axis and resting blood pressure on the y-axis
plt.plot(df['chol'], df['trestbps'])

# Add title and Labels to the plot
plt.title("Cholesterol vs Resting Blood Pressure")
plt.ylabel('Resting Blood Pressure')
plt.xlabel('Cholesterol')

# Display the Plot for Data Visualization
plt.show()
```



Task 03

Create a KNN model and train it using the given data set (train_test_split class can be used to generate training and test data set for this).

18. Import the sklearn Libraries

```
[42]: # Split the dataset into training and testing sets for model evaluation
      from sklearn.model_selection import train_test_split

      # K-Nearest Neighbors classifier for classification tasks
      from sklearn.neighbors import KNeighborsClassifier

      # Metrics for evaluating model performance
      from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

19. Features and Target Variable

```
[44]: # Features and target variable
      X = df.drop('target', axis=1)
      y = df['target']
```

20. Split the Dataset

```
[47]: # Split the dataset into training and testing sets (80% train, 20% test)
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

21. Initialize the KNN Classifier with 5 Neighbors

```
[49]: # Initialize the KNN classifier with 5 neighbors
      knn = KNeighborsClassifier(n_neighbors=5)
```

22. Train the Model on the Training Data

```
[51]: # Train the model on the training data
      knn.fit(X_train, y_train)
```

```
[51]: KNeighborsClassifier
      KNeighborsClassifier()
```

23. Predict on the Test Data

```
[53]: # Predict on the test data
      y_pred = knn.predict(X_test)
```

24. Evaluate Model Performance

```
[57]: # Evaluate model performance
      print("Accuracy:", accuracy_score(y_test, y_pred))

      Accuracy: 0.6557377049180327
```

25. Calculate Accuracy

```
[59]: # Calculate accuracy
      accuracy = accuracy_score(y_test, y_pred)
      print(f'Accuracy: {accuracy * 100:.2f}%')

      Accuracy: 65.57%
```

26. Print Classification Report

```
[61]: print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.67       0.55       0.60         29
     1       0.65       0.75       0.70         32

 accuracy          0.66
 macro avg          0.66
 weighted avg       0.66
```

27. Print Confusion Matrix

```
[63]: print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Confusion Matrix:
[[16 13]
 [ 8 24]]
```

Task 04

Add three values to make a prediction and present the predicted results and plot them in a graph with train data.

28. Adding New Data For Predication

```
[93]: # Adding new data for prediction
new_data = pd.DataFrame({
    'age': [57, 61, 43],
    'sex': [0, 1, 1],
    'cp': [1, 1, 0],
    'trestbps': [130, 156, 150],
    'chol': [236, 245, 247],
    'fbs': [0, 0, 0],
    'restecg': [0, 0, 1],
    'thalach': [174, 143, 171],
    'exang': [0, 0, 0],
    'oldpeak': [0, 0, 1.5],
    'slope': [1, 2, 2],
    'ca': [1, 0, 0],
    'thal': [2, 2, 2]
})
```

```
[95]: age = [57, 61, 43]
```

29. Predict using the Trained KNN Model

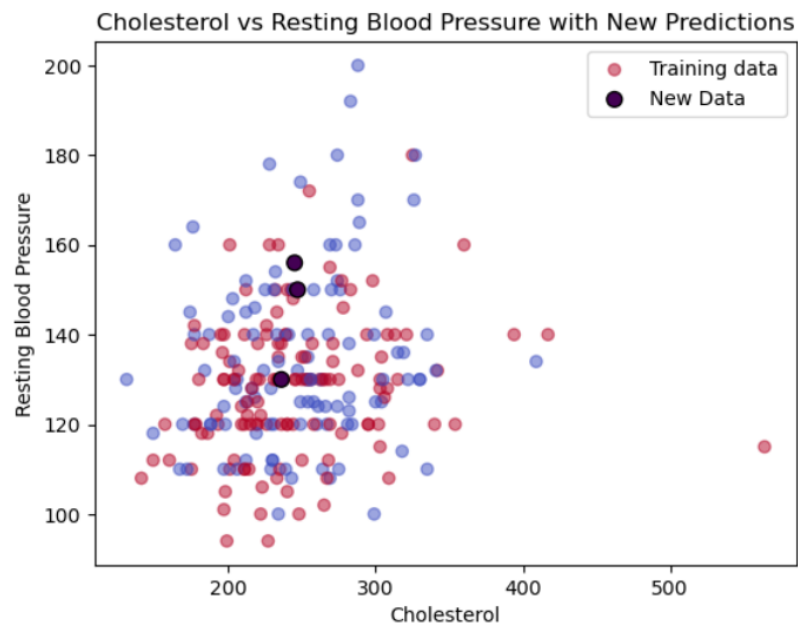
```
[97]: # Predict using the trained KNN model
predictions = knn.predict(new_data)
print("Predictions for new samples:", predictions)
```

```
Predictions for new samples: [1 1 1]
```

30. Plot the New Samples Alongside the Training Data

```
[131]: # Plot the new samples alongside the training data
plt.scatter(X_train['chol'], X_train['trestbps'], c=y_train, cmap='coolwarm', alpha=0.5, label="Training data")
plt.scatter(new_data['chol'], new_data['trestbps'], c=predictions, marker='o', s=60, label="New Data", edgecolor='black')
plt.title("Cholesterol vs Resting Blood Pressure with New Predictions")
plt.xlabel("Cholesterol")
plt.ylabel("Resting Blood Pressure")
plt.legend()
plt.show()
```

31. Display The Predicted Results



Full Code

```
[2]: # Import Pandas Libraries
import pandas as pd

[4]: # Import Numpy Libraries
import numpy as np

[6]: # Load the Dataset
df=pd.read_csv("heart.csv")

[8]: # View the Dataset
print(df)
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	\
0	63	1	3	145	233.0	1.0	0	150	0	2.3	
1	63	1	3	145	233.0	1.0	0	150	0	2.3	
2	37	1	2	130	250.0	0.0	1	187	0	3.5	
3	41	0	1	130	204.0	0.0	0	172	0	1.4	
4	56	1	1	120	236.0	0.0	1	178	0	0.8	
..	...	...	..	...	...	...	...	...	...	...	
299	57	0	0	140	241.0	0.0	1	123	1	0.2	
300	45	1	3	110	264.0	0.0	1	132	0	1.2	
301	68	1	0	144	193.0	1.0	1	141	0	3.4	
302	57	1	0	130	131.0	0.0	1	115	1	1.2	
303	57	0	1	130	236.0	0.0	0	174	0	0.0	

	slope	ca	thal	target
0	0	0	1	1
1	0	0	1	1
2	0	0	2	1
3	2	0	2	1
4	2	0	2	1
..	...	..	...	...
299	1	0	3	0
300	1	0	3	0
301	1	2	3	0
302	1	1	3	0
303	1	1	2	0

[304 rows x 14 columns]

```
[10]: # Checking for Null Values
df.isnull().sum()
```

```
[10]: age      0
sex      0
cp       0
trestbps  0
chol     1
fbs      1
restecg   0
thalach   0
exang     0
oldpeak   0
slope     0
ca        0
thal      0
target    0
dtype: int64
```

```
[12]: # Filling for Null Values
df = df.fillna({'chol': 255, 'fbs': 1})
```

```
[14]: # Again Checking Null Values
df.isnull().sum()
```

```
[14]: age      0
      sex      0
      cp      0
      trestbps  0
      chol     0
      fbs      0
      restecg   0
      thalach   0
      exang     0
      oldpeak   0
      slope     0
      ca        0
      thal      0
      target    0
      dtype: int64
```

```
[16]: # Checking for Duplicates
df.duplicated().sum()
```

```
[16]: 2
```

```
[18]: # Removing Duplicates
df = df.drop_duplicates()
```

```
[20]: # Again Checking Duplicates Values
df.duplicated().sum()
```

```
[20]: 0
```

```
[22]: # Import Matplotlib
import matplotlib.pyplot as plt
```

```
[26]: # Extract cholesterol values from the 'chol' column as a NumPy array
x=df["chol"].values
```

```
[28]: # Display the extracted cholesterol values
print(x)
```

```
[233. 250. 204. 236. 354. 192. 294. 263. 255. 168. 239. 275. 266. 211.
 283. 219. 340. 226. 247. 239. 234. 233. 226. 243. 199. 302. 212. 175.
 417. 197. 198. 177. 219. 273. 213. 177. 304. 232. 269. 360. 308. 245.
 208. 264. 321. 325. 235. 257. 216. 234. 256. 302. 231. 141. 252. 201.
 222. 260. 182. 303. 265. 309. 186. 203. 211. 183. 222. 234. 220. 209.
 258. 227. 204. 261. 213. 250. 245. 221. 205. 240. 250. 308. 318. 298.
 265. 564. 277. 197. 214. 248. 255. 207. 223. 288. 160. 226. 394. 233.
 315. 246. 244. 270. 195. 240. 196. 211. 234. 236. 244. 254. 325. 126.
 313. 211. 262. 215. 214. 193. 204. 243. 303. 271. 268. 267. 199. 210.
 204. 277. 196. 269. 201. 271. 295. 235. 306. 269. 178. 208. 201. 263.
 295. 303. 209. 223. 197. 245. 242. 240. 226. 180. 228. 149. 227. 278.
 220. 197. 253. 192. 220. 221. 240. 342. 157. 175. 286. 229. 268. 254.
 203. 256. 229. 284. 224. 206. 167. 230. 335. 177. 276. 353. 225. 330.
 230. 243. 290. 253. 266. 233. 172. 305. 216. 188. 282. 185. 326. 231.
 254. 267. 248. 197. 258. 270. 274. 164. 255. 239. 258. 188. 177. 229.
 260. 219. 307. 249. 341. 263. 330. 254. 256. 407. 217. 282. 288. 239.
 174. 281. 198. 288. 309. 243. 289. 289. 246. 322. 299. 300. 293. 304.
 282. 269. 249. 212. 274. 184. 274. 409. 246. 283. 254. 298. 247. 294.
 299. 273. 309. 259. 200. 244. 231. 228. 230. 282. 269. 206. 212. 327.
 149. 286. 283. 249. 234. 237. 234. 275. 212. 218. 261. 319. 166. 315.
 204. 218. 223. 207. 311. 204. 232. 335. 205. 203. 318. 225. 212. 169.
 187. 197. 176. 241. 264. 193. 131. 236.]
```

```
[32]: # Extract resting blood pressure values from the 'trestbps' column as a NumPy array
y=df["trestbps"].values
```

```
[34]: # Display the extracted blood pressure values
print(y)
```

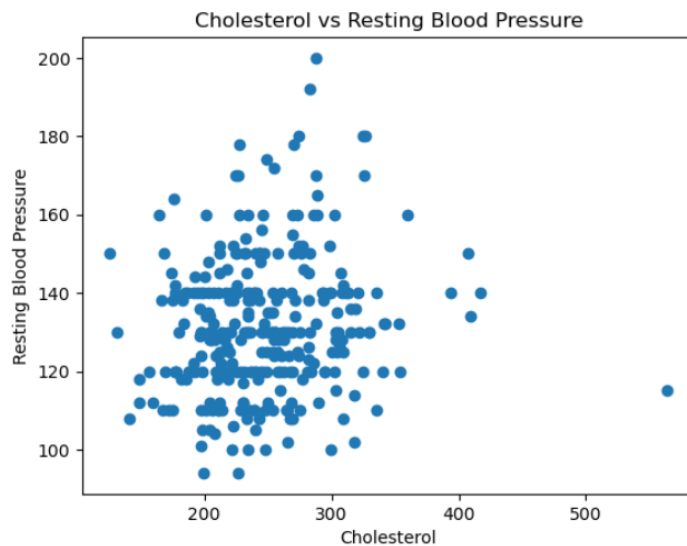
```
[145 130 130 120 120 140 140 120 172 150 140 130 130 110 150 120 120 150
 150 140 135 130 140 150 140 160 150 110 140 130 105 120 130 125 125 142
 135 150 155 160 140 130 104 130 140 120 140 138 128 138 130 120 130 108
 135 134 122 115 118 128 110 108 118 135 140 138 100 130 120 124 120 94
 130 140 122 135 125 140 128 105 112 128 102 152 102 115 118 101 110 100
 124 132 138 132 112 142 140 108 130 130 148 178 140 120 129 120 160 138
 120 110 180 150 140 110 130 120 130 120 105 138 130 138 112 108 94 118
 112 152 136 120 160 134 120 110 126 130 120 128 110 128 120 115 120 106
 140 156 118 150 120 130 160 112 170 146 138 130 130 122 125 130 120 132
 120 138 160 120 140 130 140 130 110 120 132 130 110 117 140 120 150 132
 150 130 112 150 112 130 124 140 110 130 128 120 145 140 170 150 125 120
 110 110 125 150 180 160 128 110 150 120 140 128 120 118 145 125 132 130
 130 135 130 150 140 138 200 110 145 120 120 170 125 108 165 160 120 130
 140 125 140 125 126 160 174 145 152 132 124 134 160 192 140 140 132 138
 100 160 142 128 144 150 120 178 112 123 108 110 112 180 118 122 130 120
 134 120 100 110 125 146 124 136 138 136 128 126 152 140 140 134 154 110
 128 148 114 170 152 120 140 124 164 140 110 144 130 130]
```



```
[38]: # Create a scatter plot with cholesterol on the x-axis and resting blood pressure on the y-axis
plt.scatter(df['chol'], df['trestbps'])

# Add title and labels to the plot
plt.title("Cholesterol vs Resting Blood Pressure")
plt.ylabel('Resting Blood Pressure')
plt.xlabel('Cholesterol')

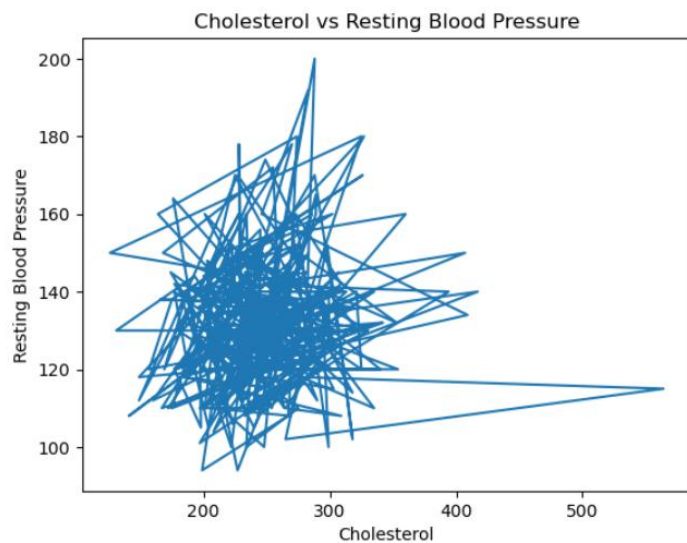
# Display the Scatter Plot for Data Visualization
plt.show()
```



```
[40]: # Create a plot with cholesterol on the x-axis and resting blood pressure on the y-axis
plt.plot(df['chol'], df['trestbps'])

# Add title and labels to the plot
plt.title("Cholesterol vs Resting Blood Pressure")
plt.ylabel('Resting Blood Pressure')
plt.xlabel('Cholesterol')

# Display the Plot for Data Visualization
plt.show()
```



```
[42]: # Split the dataset into training and testing sets for model evaluation
from sklearn.model_selection import train_test_split

# K-Nearest Neighbors classifier for classification tasks
from sklearn.neighbors import KNeighborsClassifier

# Metrics for evaluating model performance
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
[44]: # Features and target variable
X = df.drop('target', axis=1)
y = df['target']
```

```
[47]: # Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
[49]: # Initialize the KNN classifier with 5 neighbors
knn = KNeighborsClassifier(n_neighbors=5)
```

```
[51]: # Train the model on the training data
knn.fit(X_train, y_train)
```

```
[51]: KNeighborsClassifier
KNeighborsClassifier()
```

```
[53]: # Predict on the test data
y_pred = knn.predict(X_test)
```

```
[57]: # Evaluate model performance
print("Accuracy:", accuracy_score(y_test, y_pred))

Accuracy: 0.6557377049180327
```

```
[59]: # Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy * 100:.2f}%')
```

Accuracy: 65.57%

```
[61]: print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.67       0.55       0.60         29
     1       0.65       0.75       0.70         32

 accuracy          0.66
 macro avg         0.66
 weighted avg      0.66
```

```
[63]: print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Confusion Matrix:
[[16 13]
 [ 8 24]]
```

```
[93]: # Adding new data for prediction
new_data = pd.DataFrame({
    'age': [57, 61, 43],
    'sex': [0, 1, 1],
    'cp': [1, 1, 0],
    'trestbps': [130, 156, 150],
    'chol': [236, 245, 247],
    'fbs': [0, 0, 0],
    'restecg': [0, 0, 1],
    'thalach': [174, 143, 171],
    'exang': [0, 0, 0],
    'oldpeak': [0, 0, 1.5],
    'slope': [1, 2, 2],
    'ca': [1, 0, 0],
    'thal': [2, 2, 2]
})
```

```
[95]: age = [57, 61, 43]
```

```
[97]: # Predict using the trained KNN model
predictions = knn.predict(new_data)
print("Predictions for new samples:", predictions)

Predictions for new samples: [1 1 1]
```

```
[131]: # Plot the new samples alongside the training data
plt.scatter(X_train['chol'], X_train['trestbps'], c=y_train, cmap='coolwarm', alpha=0.5, label="Training data")
plt.scatter(new_data['chol'], new_data['trestbps'], c=predictions, marker='o', s=60, label="New Data", edgecolor='black')
plt.title("Cholesterol vs Resting Blood Pressure with New Predictions")
plt.xlabel("Cholesterol")
plt.ylabel("Resting Blood Pressure")
plt.legend()
plt.show()
```

